데이터베이스

D = < S, O, C >

Relation – 연산 (Operation) - 관계 해석

오병우

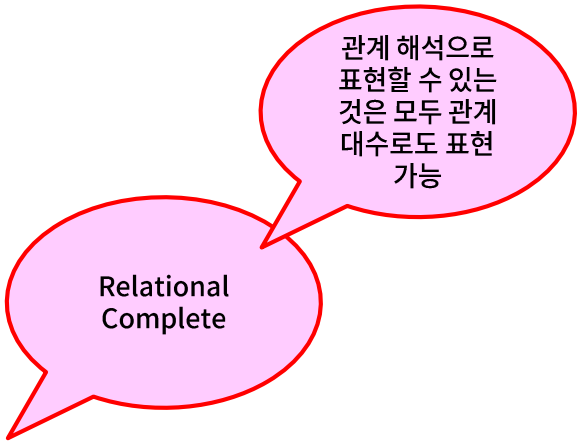
컴퓨터공학부

22일 오후 6시 시험

kit 국립금오공과대학교
Kumoh National Institute of Technology

데이터 모델

- 데이터 모델 (D)
  - D = <S, O, C>
- 연산과 데이터 언어
  - 연산 : 시스템 입장
  - 데이터 언어 : 사용자 입장
- 관계 데이터 언어
  - ⅰ. 관계 대수(relational algebra)
    - 절차 언어 : how
  - ⅱ. 관계 해석(relational calculus)
    - 비 절차 언어 : what
    - 투플 관계해석
    - 도메인 관계해석
- 관계 해석과 관계 대수는 표현이나 기능 면에서 동등



관계 해석 소개

- 관계 해석 (Relational calculus)
  - 관계형 데이터베이스에 특별히 맞춰진 응용된 프레디킷 (predicate, 술어) 해석 (calculus)
  - 질의 표현식을 위한 first-order predicate calculus (1차 술어 해석 (논리))
- 관계 대수 vs. 관계 해석
  - 유사성
    - 관계 데이터 모델의 연산 표현 방법
    - 서로 정확하게 동일함
  - 차이점

관계 대수	관계 해석
-명시적 연산의 모음 (join, union, projection, etc) -절차적 (procedural) (how) 1. join, 2. selection 3. projection	-원하는 릴레이션의 정의를 작성하기 위한 표기법 -비절자척 (nonprocedural, descriptive (서술적)) (what) -원하는 정보가 무엇이라는 것만 선언

개요

- 두 종류의 관계 해석
  - 어떤 것을 변수로 둘지: Tuple 또는 Domain
- Tuple 해석 (TRC: Tuple Relational Calculus)
  - 투플 변수: 투플에 대한 범위
    - 지정된 릴레이션의 한 투플만을 그 값으로 취할 수 있는 변수
- Domain 해석 (DRC: Domain Relational Calculus)
  - 도메인 변수: 도메인 요소에 대한 범위 (= 필드 값)
    - 지정된 애트리뷰트의 도메인의 한 원소 값만을 값으로 취하는 변수
- Variable (변수)
  - 변하는 수: 저장된 값이 변함
  - 한 번에는 한 개만 저장

변수가 tuple

struct R t;

struct R {
int aa;
char bb;
}

변수가 attribute

int a;
char b;

투플 변수 및 한정 애트리뷰트

- 투플 변수 (Tuple Variables)
  - 지정된 릴레이션의 한 투플만을 그 값으로 취할 수 있는 변수
  - 투플 변수(tuple variable) 또는 범위변수(range variable): t
  - 범위식 (range formula) : R(t)
    - t는 R의 투플 변수
  - R : t의 범위 릴레이션 (range relation)

```
struct R t;
```

```
struct R {
 int aa;
 char bb;
}
```

- 한정 애트리뷰트(qualified attribute) : t.A 또는 t[A]
  - 투플 변수 t가 나타내는 투플의 어떤 애트리뷰트 A의 값
  - 예제
    - Student(s)
    - s.Sno

```
t.aa
t.bb
```

1차 (First-order) 술어 해석 (Predicate Calculus)

이해 안되면 skip

- Predicate (서술, 술어, 서술어, 프레디킷)
  - 값이 참 또는 거짓인 함수
  - 객체 인수를 TRUE 또는 FALSE로 매핑하는 함수
- first-order predicate calculus (1차 술어 해석)
  - 인수에 predicate이 오면 안됨 (e.g., Predicate(func1(3)) 안됨)
    - i.e., 값만 와야함 (e.g., Predicate(3) 가능)
  - 문장의 주어가 개별 개체 (소크라테스는 죽는다)
  - 정량자(Quantifier)가 변수에만 적용되고 predicate나 함수에 대해서는 허용되지 않음
    - (∀x)P(x)
  - 서술어, 함수, 변수, 상수, 한정자, 논리적 연결자 등으로 구성된 Symbol로 표현
- second-order 또는 higher order predicate calculus
  - 인수에 predicate 허용
  - 주어가 술어(predicate)로 구성
    - 죽는다는 것은 비극이다, (∀P)P(x)

Second -order

```
bool Predicate(int a)
{
    if (a > 0 && a < 10)
        return true;
    else
        return false;
}
```

```
Predicate( func1(b) )
```

Tuple 관계 해석

- 원하는 릴레이션을 tuple calculus 표현식으로 정의하는 표기법
- 질의 : { t | P(t) }으로 표현
  - P(t)는 tuple 변수 t에 대한 공식 (formula)
    - Calculus의 표현식을 공식(formula)이라고 부름
- 결과
  - 공식 P(t)가 TRUE로 평가되는 모든 투플 t의 집합
  - 우리가 얻으려는 것은 t의 집합임, P(t)는 t가 될 수 있는 값을 제한함
- 공식 (Formula)
  - 재귀적으로 (Recursively) 정의됨 (중첩 (nested) 공식)
    - 간단한 원자 공식으로 시작
    - 관계 또는 값 비교에서 튜플 가져오기
    - 논리적 접속사 (¬, ∧, ∨, =)를 사용하여 더 크고 더 나은 수식 만들기

= not

```
bool Predicate(int a)
{
    if (a > 0 && a < 10)
        return true;
    else
        return false;
}
```

```
{ t | Predicate(t) }
= { 1,2,3,4,5,6,7,8,9 }
```

공식 (Formulas)

atom, 원자 표현식 (atomic expression), 명제 (proposition) 라고도 부름

- Atomic formula (원자 공식: 더 이상 쪼갤 수 없는 formula)
  - R(t)
    - t : 투플 변수, R : t의 범위 릴레이션
  - t.A θ u.B
    - t, u : 투플 변수, A, B : t와 u에 대한 한정 애트리뷰트
    - θ : 비교 연산자(=, ≠, <, ≤, >,≥)
  - t.A θ c
    - A : 투플 변수 t에 대한 한정 애트리뷰트, c : 상수
  - Atom의 결과는 반드시 참(True) 또는 거짓(False)
- 다음은 공식이 될 수 있음:
  - 원자 공식 (An atomic formula)
  - p와 q가 공식인 경우 ¬ p, p∧q, p∨q
  - 변수 t가 투플 변수인 경우 (∃t)P(t)
  - 변수 t가 투플 변수인 경우 (∀t)P(t)

속박 변수 및 자유 변수

- Quantifiers (정량자)
  - ∀ : 전칭 정량자(Universal quantifier) – “for all”
  - ∃ : 존재 정량자(Existential quantifier) – “there exists”
- Bound Variable (속박 변수)
  - Formula에 ∃x 또는 ∀x를 포함하고 있다면 x는 bound variable
    - cf.) bound variable이 아닌 것은 free variable
  - 막대(|)의 오른쪽에만 있는 변수는 모두 속박(∀, ∃)되어야 함
- Free Variable (자유 변수)
  - Quantifiers (∀ and ∃)로 한정되지 않는 투플 변수
  - 막대(|)의 왼쪽에 존재
  - 막대(|)의 왼쪽에 있는 free variable이 막대의 오른쪽에서 속박 (∀, ∃) 되면 안됨

WFF: Well-Formed Formula

- 정형식(WFF, Well-formed formula)
  - Atom, 논리 연산자 (∧, ∨, ¬), quantifier (∀, ∃)가 다음 규칙에 따라 결합된 식
    - ① 모든 atomic formula(atom)는 WFF
    - ② F가 WFF이면, (F)와 ¬F도 WFF
    - ③ F와 G가 WFF이면, F∧G와 F∨G도 WFF
    - ④ F(t)가 WFF이면, ∀t(F(t))와 ∃t(F(t))도 WFF
    - ⑤ 위의 규칙만을 반복 적용해서 만들어진 식은 WFF
  - WFF의 예제
    - s.Sno = 100
    - c.Cno ≠ e.Cno
    - s.Sno = e.Sno ∧ e.Cno ≠ c.Cno
    - (∃e)(e.Sno = s.Sno ∧ e.Cno = 'C413')

Tuple 관계 해석 표현식

- 형식
  - { t1.A1, t2.A2, ⋯, tn.An | F(t1, ⋯ tn, tn+1, ⋯, tn+m) }
  - ti : 투플 변수
    - '|'의 왼쪽에 있는 변수 t는 공식 P(t)에서 반드시 자유 변수여야함
    - 즉, 다른 모든 tuple variable들은 quantifier를 사용한 bound variable 이어야 함
  - F(t1,⋯, tn, tn+1,⋯, tn+m): ti가 연관된 WFF로 조건을 명세
  - 결과 리스트 (Target list)
    - 막대 (|) 왼편에 나온 한정 애트리뷰트들 (쉼표로 구분)
    - 막대 (|) 오른편에 명세된 조건을 만족하는 결과로 추출 됨

Targe t list

WFF

막대의 왼쪽에 있는 변수는 오른쪽에서 속박(∀,∃)되면 안됨

- examples
  - {s.Sname | STUDENT(s) } STUDENT중 Sname만 꺼내라
  - {s.Sname | STUDENT(s)∧s.Dept='컴퓨터' }
  - {s.Sname, s.Dept | STUDENT(s)∧(∃e)(ENROL(e)∧ s.Sno=e.Sno∧e.Grade='A')}

예제

막대의 왼쪽에 있는 변수는 오른쪽에서 bound(속박) (∀,∃)되면 안됨 (free)

- 과목 C413에서 성적이 A인 학생의 학번을 모두 검색하라.
  - {e.Sno | ENROL(e) ∧ e.Cno='C413' ∧ e.Grade='A' }
- C413을 등록한 학번을 검색하라.
  - { e.Sno | ENROL(e) ∧ e.Cno='C413' }
- C413을 등록한 학생의 이름 및 학과를 검색하라.
  - {s.Sname, s.Dept | STUDENT(s)∧(∃e)(ENROL(e)∧ s.Sno=e.Sno∧ e.Cno='C413')}
- C413을 등록한 학생 이름, 학과, 과목 번호를 검색하라.
  - { s.Sname, s.Dept, e.Cno | STUDENT(s) ∧ ENROL(e) ∧ s.Sno=e.Sno ∧ e.Cno='C413' }
- 모든 과목에 등록한 학생의 이름을 전부 검색하라.
  - { s.Sname | STUDENT(s) ∧
    (∀c)(∃e)(COURSE(c) ∧ ENROL(e) ∧ e.Sno=s.Sno ∧ e.Cno=c.Cno) }
- 과목 C413에 등록하지 않은 학생의 이름 전부를 검색하라.
  - {s.Sname | STUDENT(s) ∧ (¬ ∃e)(ENROL(e) ∧ s.Sno=e.Sno ∧ e.Cno='C413') }

오른쪽에서 연산에 사용되면 속박되어야 함

왼쪽에 있으면 bound되면 안됨 (free)

Domain 관계 해석

- 원하는 릴레이션을 domain calculus 표현식으로 정의하는 표기법
- Domain 변수
  - 지정된 애트리뷰트의 도메인의 한 원소만을 값으로 취하는 변수
    - 편의상 애트리뷰트 이름 앞에 x 붙임: xSno, xSname, …
  - 범위식을 사용하여 도메인 선언
    - STUDENT(xSno, xSname, xDept, xYear)
- Atomic formula
  - ① R(x1,x2,⋯,xn)
    - xi : 도메인 변수, R : xi의 range relation (범위 릴레이션)
    - <x1,x2,⋯,xn>에 해당하는 값의 리스트는 릴레이션 R의 투플
  - ② x θ y
    - x, y : 도메인 변수, θ: 비교 연산자(=, ≠, <, ≤, >,≥)
  - ③ x θ c
    - x : 도메인 변수, θ: 비교 연산자, c : x가 정의된 도메인 값의 상수
  - Atomic formula의 실행 결과는 반드시 참(True) 또는 거짓(False)

Domain 관계 해석 표현식

- 형식

Target list

WFF

  - { x1,x2,⋯,xn | F(x1,⋯,xn,xn+1,⋯,xn+m) }
    - xi: 도메인변수
    - F(x1,⋯,xn,xn+1,⋯,xn+m) : xi에 대한 WFF
  - 결과 리스트 (Target list)
    - 막대 (|) 왼편에 나온 도메인 변수들 (쉼표로 구분)
    - 막대 (|) 오른편에 명세된 조건을 만족하는 도메인 값으로 만들어지는 투플
  - example
    - { xSname | STUDENT(xSno, xSname, xYear, xDept)}
    - { xSname | (∃xDept)(STUDENT(xSno, xSname, xYear, xDept)∧ xDept='컴퓨터') }
    - { xSno, xDept | STUDENT(xSno, xSname, xYear, xDept) ∧ (∃xxSno)(∃xGrade)(ENROL(xxSno, xCno, xGrade, xMidterm, xFinal) ∧ xSno=xxSno∧xGrade='A') }

Qunatifier 를 앞에 쓰는 방법

Prenex normal form (프리닉스 정규형)

예제

막대의 왼쪽에 있는 변수는 오른쪽에서 속박(∀,∃)되면 안됨

오른쪽에서 연산에 사용되면 속박되어야 함

오른쪽에서 연산에 사용되면 속박되어야 함

- 컴퓨터학과 3학년 이상의 학생 이름을 검색하라.
  - {xSname | (∃xYear)(∃xDept)(STUDENT(xSno, xSname, xYear, xDept) ∧ xYear ≥ 3 ∧ xDept='컴퓨터') }
- 과목 C413에서 성적이 A인 학생의 학번을 모두 검색하라
  - { xSno | (∃xCno)(∃xGrade)(ENROL(xSno, xCno, xGrade, xMidterm, xFinal) ∧ xCno= 'C413' ∧ xGrade='A') }
- 기말 성적이 90점 이상인 학생의 학번과 이름을 검색하라.
  - {xSno, xSname | STUDENT(xSno,xSname,xYear,xDept) ∧ (∃xFinal)(∃xxSno) (ENROL(xxSno, xCno, xGrade, xMidterm, xFinal)∧ xSno=xxSno ∧ xFinal ≥ 90) }
- 과목 C324에 등록하지 않은 학생의 이름을 검색하라.
  - { xSname | (∃xSno)(STUDENT(xSname, xSno, xYear, xDept) ∧ (¬ ∃xxSno) (∃xCno) (ENROL(xxSno, xCno, xGrade, xMidterm, xFinal) ∧ xSno=xxSno ∧ xCno='C324')) }

성적이 A인 학생의 이름을 검색하라 (equijoin, distinct 사용)

- 투플 관계 해석
  - {s.Sname | STUDENT(s) ∧ (∃e)(ENROL(e) ∧ s.Sno = e.Sno ∧ e.Grade = 'A')}
- 도메인 관계 해석
  - {xSname | (∃xSno)(STUDENT(xSno, xSname, xYear, xDept) ∧ (∃xxSno)(∃xGrade)(ENROL(xxSno, xCno, xGrade, xMidterm, xFinal) ∧ xSno = xxSno ∧ xGrade = 'A'))}
- SQL
  - SELECT DISTINCT Sname
    FROM STUDENT, ENROL
    WHERE STUDENT.Sno = ENROL.Sno and Grade = 'A'
- 결과 relation

Sname
나수영
정기태
송병길

4/9

학생 (STUDENT)

학번 (Sno)	이름 (Sname)	학년 (Year)	학과 (Dept)
100	나수영	4	컴퓨터
200	이찬수	3	전기
300	정기태	1	컴퓨터
400	송병길	4	컴퓨터
500	박종화	2	산공

과목 (COURSE)

과목번호 (Cno)	과목이름 (Cname)	학점 (Credit)	학과 (Dept)	담당교수 (PRname)
C123	프로그래밍	3	컴퓨터	김성국
C312	자료 구조	3	컴퓨터	황수관
C324	화일 구조	3	컴퓨터	이규찬
C413	데이터베이스	3	컴퓨터	이일로
E412	반도체	3	전자	홍봉진

등록 (ENROL)

학번 (Sno)	과목번호 (Cno)	성적 (Grade)	중간성적 (Midterm)	기말성적 (Final)
100	C413	A	90	95
100	E412	A	95	95
200	C123	B	85	80
300	C312	A	90	95
300	C324	C	75	75
300	C413	A	95	90
400	C312	A	90	95
400	C324	A	95	90
400	C413	B	80	85
400	E412	C	65	75
500	C312	B	85	80

성적이 A인 학생의 이름을 검색하라 (SQL)

- Join on 및 distinct 사용 (Relational Algebra 표현에 가까움) (권장)
  - SELECT **DISTINCT** s.Sname
    FROM STUDENT s **JOIN** ENROL e **ON** s.Sno = e.Sno
    WHERE e.Grade = 'A'
- Join using 및 distinct 사용 (Relational Algebra 표현에 가까움)
  - SELECT **DISTINCT** Sname
    FROM STUDENT **JOIN** ENROL **USING** (Sno)
    WHERE Grade = 'A'
- Exists 사용 (Tuple Relational Calculus 표현(∃e)에 가까움)
  - SELECT s.Sname
    FROM STUDENT s
    WHERE **EXISTS** (SELECT 1
    FROM ENROL e
    WHERE e.Sno = s.Sno AND e.Grade = 'A')

select 1: 존재 여부만 판단

select * 또는 select e.sno 써도 됨

성적이 A인 학생의 이름을 검색하라 (equijoin, distinct 사용)

관계 대수

- 관계 대수
  - $\Pi_{Sname}(\sigma_{Grade = 'A'}(STUDENT \bowtie_{Sno=Sno} ENROL))$
- 데이터 언어식 표현
  - (STUDENT [Sno = Sno] ENROL WHERE Grade = 'A')[Sname]
- SQL
  - SELECT DISTINCT Sname
    FROM STUDENT, ENROL
    WHERE STUDENT.Sno = ENROL.Sno and Grade = 'A'

Natural Join이면 STUDENT JOIN ENROL

- 결과 relation

Sname
나수영
정기태
송병길

학생 (STUDENT)

학번 (Sno)	이름 (Sname)	학년 (Year)	학과 (Dept)
100	나수영	4	컴퓨터
200	이찬수	3	전기
300	정기태	1	컴퓨터
400	송병길	4	컴퓨터
500	박종화	2	산공

과목 (COURSE)

과목번호 (Cno)	과목이름 (Cname)	학점 (Credit)	학과 (Dept)	담당교수 (PRname)
C123	프로그래밍	3	컴퓨터	김성국
C312	자료 구조	3	컴퓨터	황수관
C324	화일 구조	3	컴퓨터	이규찬
C413	데이터베이스	3	컴퓨터	이일로
E412	반도체	3	전자	홍봉진

등록 (ENROL)

학번 (Sno)	과목번호 (Cno)	성적 (Grade)	중간성적 (Midterm)	기말성적 (Final)
100	C413	A	90	95
100	E412	A	95	95
200	C123	B	85	80
300	C312	A	90	95
300	C324	C	75	75
300	C413	A	95	90
400	C312	A	90	95
400	C324	A	95	90
400	C413	B	80	85
400	E412	C	65	75
500	C312	B	85	80